Edge AI Lab 2 Report: Part 3 Calibration-Free Quantization

Pham Minh Long 龍志　　Student ID: 314540080

Method

I reproduce AdaptQuant, so I do not introduce the method again. Below are only my implementation choices and the parts the paper leaves open.

- Setting: Qwen/Qwen3-4B in fp16 on one NVIDIA RTX A5000. Every nn.Linear in the 36 decoder blocks is fake-quantized on both operands (MXFP4 or MXFP6 per 16 × 32 tile), the weights once at load time and the input activations on every forward pass.
- Thresholds: the paper does not publish its 14 threshold pairs, so I ran its Algorithm 1 myself on the same reference model, Llama-3.1-8B, with one WikiText-2 train sequence of 2048 tokens. As in the paper, the search ends with $t_2 = \mathrm{CAP}$ everywhere, so only MXFP4 and MXFP6 are used.
- Calibration-free: the thresholds are fixed numbers in thresholds.json, derived once on a different model. Quantizing Qwen3-4B only reads its own weights, and activation formats are chosen per tile at run time.
- Kernel: the tile quantizer is a CUDA kernel (src/mx_tile_fake_quant.cu, one thread block per tile, one warp per 1 × 32 block) with a PyTorch fallback. It is about 30 times faster than the PyTorch version on a 2048 × 9728 activation.

Result

Final PPL: 14.20 on WikiText-2 with Qwen3-4B (paper: 13.98).

Method	Weight bits	Activation bits	PPL (ours)	PPL (paper)
FP16	16.00	16.00	13.64	13.66
MXFP4	4.25	4.25	18.57	18.20
NVFP4	4.50	4.50	13.93	13.96
AdaptQuant (ours)	5.29	5.32	14.20	13.98

Table 1: Qwen3-4B, weights and activations quantized. Bits include the shared scale.

Operand	MXFP4 tiles	MXFP6 tiles	Bits, Qwen3-4B	Bits, Llama-3.1-8B
Weights	47.78%	52.22%	5.29	5.25
Activations	46.27%	53.73%	5.32	5.54

Table 2: Tile formats chosen by AdaptQuant with the derived thresholds.

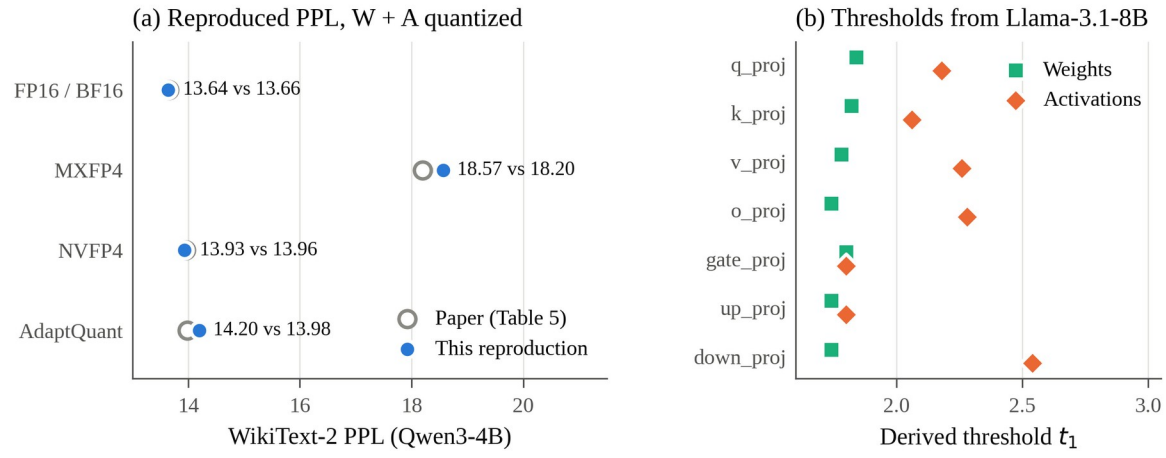


Figure 1: (a) Reproduced PPL next to the paper. (b) Derived t1 per projection type.

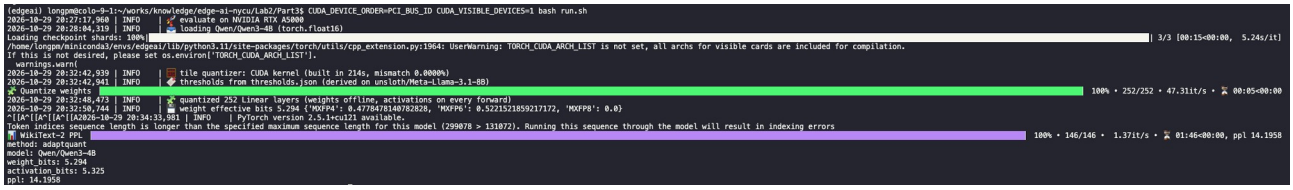


Figure 2: Terminal output of bash run.sh on one RTX A5000.

Observation and Discussion

Important observations

- The baselines match the paper: FP16 13.64 vs 13.66 and NVFP4 13.93 vs 13.96. So my evaluation protocol is the same as the paper's.
- My AdaptQuant is 0.22 PPL above the paper while it spends more weight bits (5.29 vs 5.14).
- Activation thresholds are higher than weight thresholds (Figure 1b), with down_proj the highest (2.54). Activation tiles have larger stat values, yet a similar share ends up in MXFP6 (53.7% vs 52.2% for weights).

How the quantization method affects model quality

- Uniform MXFP4 costs 4.93 PPL over FP16. AdaptQuant wins back 4.37 of it (89%) by moving about half of the tiles to MXFP6.
- The gain comes from tiles with a high stat: they hold many small values that fall under half of the smallest FP4 value and are flushed to zero. MXFP6 keeps them.
- AdaptQuant lands at NVFP4 quality (14.20 vs 13.93) because the derivation keeps each layer's error at the NVFP4 level. It does not beat NVFP4 in the paper either (13.98 vs 13.96).

Difficulties during implementation

- The thresholds are not given in the paper, so I implemented the whole derivation (weighted tile error, NVFP4 reference, prefix sums over the grid, per-layer constraint). The official meta-llama repository is gated, so I used the public copy unsloth/Meta-Llama-3.1-8B.
- My first solver picked t2 below CAP. Many (t1, t2) pairs had the same cost because no reference tile had a stat above t2, and my tie-break picked the smaller t2. Breaking ties toward more MXFP4 and then toward a larger t2 gives t2 = CAP everywhere, as in the paper, with the same PPL.
- The CUDA kernel and the PyTorch version agree on a random test matrix, but on the full model they give 14.1958 and 14.1925. The tile sum of squares is added in a different order, so a few activation tiles that sit exactly on a threshold change format.

Possible reasons for performance degradation

- My uniform MXFP4 is also 0.37 PPL worse than the paper, while NVFP4 matches. So part of the gap probably comes from my MX quantizer (scale or rounding details) and not from format selection. I did not compare it against an official MX library.
- My thresholds give 5.25 weight bits on Llama-3.1-8B, the paper reports 5.08. Details the paper does not fix (which sequence, bf16 or fp16 in the derivation, the exact NVFP4 reference) change the thresholds, and the tightest layer is only +0.017 dB above NVFP4, so small changes move t1.
- I evaluate in fp16 so the code runs on a T4, while the paper uses BF16 on an RTX 5090.

Limitations and possible improvements

- The quantization is simulated, so there is no real memory saving or speedup, and I only measured WikiText-2 PPL on one model.
- To improve, I would check the MX quantizer against the OCP reference code, derive the thresholds on more than one sequence, and try the optional bit target of Algorithm 1 to trade bits for PPL.